Smart City Waste Management System

1. Introduction & SDG Background

The Smart City Waste Management System is a comprehensive Java-based simulation designed to address urban sanitation challenges. It aligns directly with the UN's Sustainable Development Goal (SDG) 11: Sustainable Cities and Communities. By optimizing waste collection routes and providing real-time data analytics, the system aims to reduce the carbon footprint of sanitation fleets while ensuring cities remain clean and visually appealing.

2. Problem Statement

Traditional municipal waste collection relies on static, predetermined routes regardless of the actual fill levels of the bins. This inefficiency leads to two major issues: overflowing bins in high-traffic areas (causing health hazards and visual pollution) and unnecessary fuel consumption when trucks are dispatched to collect nearly empty bins. There is a critical need for a dynamic, data-driven approach.

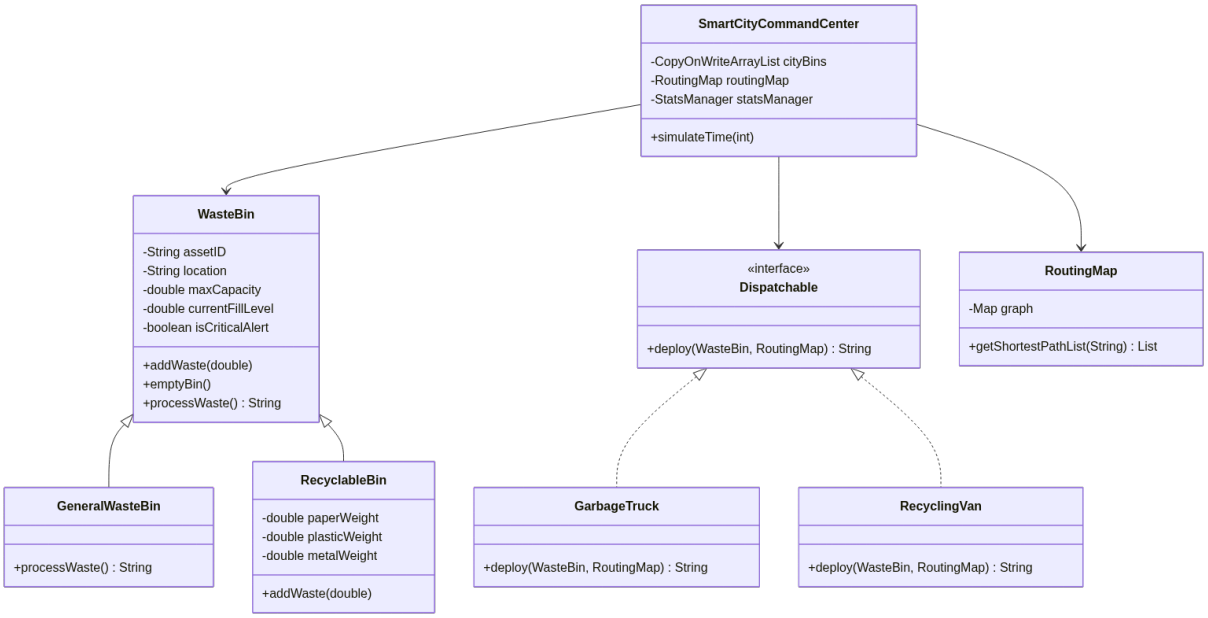
3. System Objectives

- Monitor live fill levels of both General Waste and Recyclable bins.
- Trigger automated, real-time dispatches only when bins reach a critical threshold (>85%).
- Optimize travel distances using graph theory (Dijkstra's Algorithm) to find the shortest path.
- Provide visual analytics regarding waste generation trends, geographic hot-spots, and recyclable composition.

Smart City Waste Management System

4. OOP Design (UML Class Diagram)

The system strictly adheres to Object-Oriented principles. Inheritance is utilized through the WasteBin base class and its specific derivatives (GeneralWasteBin, RecyclableBin). Polymorphism is demonstrated via the Dispatchable interface, implemented by GarbageTruck and RecyclingVan, allowing the SimulationEngine to deploy different vehicles identically.



Smart City Waste Management System

5. Implementation Details

The application is built entirely in Java (Swing/AWT) with zero external dependencies. Key features include:
- Routing Graph: A custom Adjacency List representing Cyberjaya (Depot, DPULZE Mall, MMU, etc.) utilizing Dijkstra's Algorithm.
- Multi-threading: A javax.swing.Timer drives background auto-play simulation while retaining UI responsiveness.
- Data Persistence: Configuration and Lifetime Statistics are saved locally via CSV and Properties files.
- Custom Rendering: Line Charts, Bar Charts, Pie Charts, and the Live Tracking Node Map are rendered natively using Graphics2D.

6. Testing & Sample Outputs

Testing was conducted via a highly accelerated time-lapse simulation (e.g., +1 MONTH skip). The custom Line Chart correctly visualized 30 days of cumulative waste data, appropriately reflecting randomly injected Weather events (Heavy Rain reducing waste) and Weekend Events (spiking waste by 50%). Furthermore, active trucks were visually verified moving node-by-node on the map canvas exactly as dictated by the shortest-path algorithm.

7. Discussion & Limitations

While the simulation successfully models a smart city, there are limitations. The routing map is currently hardcoded to 7 nodes in Cyberjaya. A real-world application would require integration with a live GPS/Maps API (e.g., Google Maps) to track real-time traffic and exact coordinate geography. Additionally, bin fill levels are simulated via random generation rather than hardware IoT sensors.

8. Conclusion

The Smart City Command Center effectively proves that combining Object-Oriented design with algorithmic graph theory can dramatically optimize urban waste management. The system is highly scalable, visually informative, and directly addresses the core mission of SDG 11 by promoting sustainable, data-driven city infrastructure.